

**EDGE INTELLIGENCE LABORATORY
MACSE604
ASSESSMENT 4**

**SUBMITTED BY
SHIBU P
25MML0042**

TABLE OF CONTENTS

S.No	Description	Page No
1	Flow Diagram of the use case	3
2	Sources of logs	4
3	Collecting logs	4
4	Centralized Storage	4
5	Data Processing	5
6	Training and Predicting with ML models	5
7	Data Analysis	5
8	Visualization	6
9	System for alerts	6
10	Actions and Insights	6

PLAN ON LOG DATA ANALYTICS

1. Use Case (Flow Diagram):

Real-Time Log Monitoring and Alerting in E-Commerce Applications

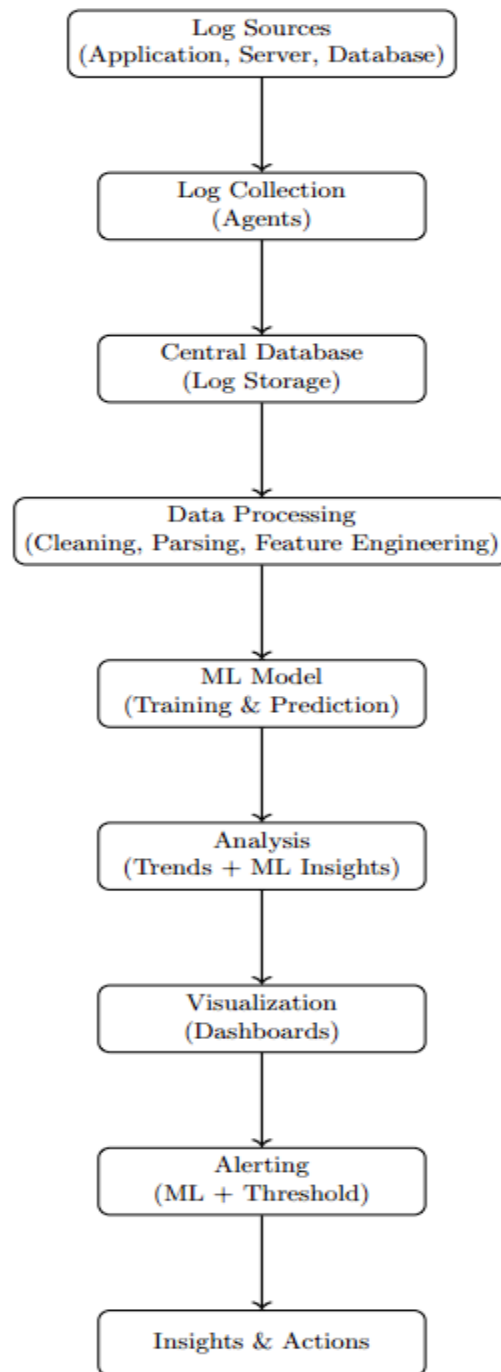


Fig 1: Flow diagram of the E-commerce log analysis

Explanation of the Flow:

2. Sources of Logs:

Log sources are the parts of an e-commerce system that record information about how users use the system and how it works. These include servers, databases, web apps, and backend services data. There are logs made every time a user logs in, searches for products, adds things to their cart, or pays. For instance, things like "login success," "product viewed," or "payment failed" are logged. Also, system logs are made that show things like server errors and how requests are handled. Most of the time, these logs are not organized and are spread out over different systems. Because these log sources are created all the time and in large amounts, it is important to find and keep track of them for effective monitoring and analysis.

3. Collecting Logs (Agents):

Filebeat and Fluentd are two examples of log collection agents that keep collecting logs in real time. Machine Learning Models are integrated with agents which might also do light preprocessing, like filtering out logs that aren't useful or tagging events that are important. This makes sure that only useful data goes into the ML pipeline. For instance, logs about user errors and transactions can be given more weight. Efficient log collection makes sure that the ML model always gets the right data for both training and making predictions in real time.

4. Centralized Storage (Database):

Elasticsearch or another centralized database stores all the logs that have been collected. In a system that uses machine learning, this database is also used to train models. We use historical log data to make datasets that help the model learn how things work. A classification model can be trained using logs of past transactions that were both successful and unsuccessful. For machine learning tasks, the database needs to be able to store a lot of data and make it easy to search through it quickly.

5.Data Processing(cleaning,parsing, and feature engineering)

This step is more important with ML integration. Along with cleaning and parsing logs, feature engineering is done to get useful features for the model. You can make features like the number of requests, the number of errors, the length of the session, and the time it takes to respond. You could turn logs into numbers, like "number of failed payments in the last 5 minutes." People use these features to teach machine learning models. Preprocessing the right way makes sure that the model can learn well and make accurate predictions.

6. Training and predicting with ML models:

This is when machine learning models are first used. Historical log data is used to train models like anomaly detection algorithms, classification models, and time-series forecasting models. For instance, we can train a model to guess whether a transaction is likely to fail based on what has happened in the past. When the system is running in real time, new logs are sent to the trained model to make predictions. The model marks something as an anomaly if it sees something strange, like a sudden rise in errors. This makes the system better at finding problems before they happen instead of after they happen.

7.Data Analysis (with ML Insights to make it better):

ML predictions now drive data analysis. The trained model lets the system find patterns and outliers on its own, instead of just finding trends by hand. For instance, the ML model might notice small changes in how users act that could mean the system is about to break. It can also put logs into groups like normal, warning, or critical. This makes analysis more accurate and faster, which lets the system handle a lot of data quickly and give more information.

8. Visualization (Dashboards with ML Output):

Matplotlib and seaborn are two examples of visualization libraries that now show both traditional metrics and machine learning predictions. Dashboards can show scores for unusual events, predicted failures, and levels of risk. For instance, a graph might show predicted spikes in payment failures before they actually happen. This helps developers and stakeholders understand how the system works now and in the future, which lets them make decisions ahead of time.

9. System for alerts (ML-Based Alerts):

ML features make the alerting system better. Alerts can be based on ML predictions instead of just fixed thresholds. If the model thinks there's a good chance the system will fail, for instance, it sends out an alert before the problem happens. This makes alerts more accurate and less likely to be false positives. ML-based alerts make sure that problems are dealt with more quickly and intelligently.

10. Actions & Insights (Smart Choices):

Analytics and ML forecasts drive insights in the last phase. Instead of waiting for failures, developers might take proactive measures based on anticipated problems. For instance, the team can scale resources or move to a backup system ahead of time if the model indicates an increase in payment failures. Both system dependability and user experience are enhanced by this. The machine learning model can be retrained with fresh data over time to continuously enhance performance, increasing the system's intelligence and adaptability.